FACE EXPRESSION RECOGNITION

CNN-Based Image Classification Project

Project Documentation & Technical Report

Environment	Google Colab
Framework	TensorFlow / Keras
Hardware	NVIDIA T4 GPU (as recorded in notebook)
Dataset source	Kaggle via kagglehub
Dataset identifier	aadityasinghal/facial-expression-dataset
Input image size	48 × 48 grayscale
Batch size	32
Recorded validation accuracy	79.42%

Prepared from the uploaded Google Colab notebook. Numerical results and implementation details in this report are based on the notebook's recorded cells and outputs.

1. Executive Summary

The project is intended to build a facial-expression recognition system using a Convolutional Neural Network (CNN). The notebook downloads the Kaggle dataset using kagglehub, prepares image datasets with TensorFlow, builds a multi-block CNN, trains it with callbacks, evaluates the model, generates a classification report and confusion matrix, saves the trained model, and includes a single-image prediction workflow.

The recorded evaluation produced a validation accuracy of 79.42% and a validation loss of 0.5087. However, an important implementation issue was recorded: the dataset was passed to image_dataset_from_directory at the dataset root, where the immediate folders were **train** and **test**. Consequently, TensorFlow detected only two classes—**test** and **train**—instead of the seven facial-expression folders inside those directories. Therefore, the 79.42% result should not be described as seven-expression recognition accuracy for the current notebook run.

2. Project Objective

The intended objective is to classify a face image into one of seven expressions: happy, fear, disgust, sad, surprise, angry, or neutral. The project uses CNNs because convolutional layers can learn spatial image features such as edges, textures, shapes, and higher-level facial patterns.

3. Technologies and Tools

Tool / Library	Purpose
Google Colab	Notebook execution environment
KaggleHub	Download the Kaggle dataset
Python	Project programming language
TensorFlow / Keras	Dataset pipeline, CNN model, training and inference
NumPy	Numerical arrays and prediction processing
Pandas	Imported for data handling
Matplotlib	Visualization
Seaborn	Confusion-matrix visualization
Scikit-learn	Classification report and confusion matrix

4. Dataset Description

The notebook downloads **aadityasinghal/facial-expression-dataset** through kagglehub. The downloaded dataset contains separate train and test directory trees. The notebook's directory walk recorded 35,887 image files in total.

Expression	Train images	Test images	Total
happy	7,215	1,774	8,989
fear	4,097	1,024	5,121
disgust	436	111	547
sad	4,830	1,247	6,077
surprise	3,171	831	4,002
angry	3,995	958	4,953
neutral	4,965	1,233	6,198

Expression	Train images	Test images	Total
Total	**28,709**	**7,178**	**35,887**

These counts come from the notebook's directory inspection. They show that the underlying dataset does contain seven expression folders, but the subsequent TensorFlow loading cell did not point to the nested class directory.

5. Data Loading and Preprocessing

The notebook sets IMG_SIZE to (48, 48), batch size to 32, and random seed to 42. Images are requested in grayscale. The intended training/validation split is 80/20 using TensorFlow's validation_split mechanism.

The CNN itself applies pixel rescaling by 1/255 and performs data augmentation using horizontal flipping, random rotation, random zoom, and random contrast. Dataset prefetching is also configured with AUTOTUNE.

6. CNN Architecture

Stage	Layers / Operation	Main role
Input	48×48×1 + Rescaling	Normalize grayscale image
Augmentation	Flip, Rotation, Zoom, Contrast	Improve robustness
CNN Block 1	Conv2D 32 → BatchNorm → Conv2D 32 → MaxPool → Dropout 0.25	Learn low-level features
CNN Block 2	Conv2D 64 → BatchNorm → Conv2D 64 → MaxPool → Dropout 0.25	Learn intermediate features
CNN Block 3	Conv2D 128 → BatchNorm → Conv2D 128 → MaxPool → Dropout 0.30	Learn higher-level features
Classifier	Flatten → Dense 256 → BatchNorm → Dropout 0.50	Convert features to class evidence
Output	Dense(number of classes, softmax)	Class probability output

7. Model Compilation and Training

The model is compiled with the Adam optimizer using a learning rate of 0.001. The loss function is sparse categorical cross-entropy and accuracy is used as the main metric.

Training is configured for up to 40 epochs. Three callbacks are used: EarlyStopping with patience 7 and restored best weights, ReduceLROnPlateau with factor 0.5 and minimum learning rate 1e-6, and ModelCheckpoint to save the best validation-accuracy model.

8. Evaluation Results Recorded in the Notebook

Metric	Recorded result
Validation samples	7,177
Validation loss	0.5087
Validation accuracy	79.42%
Classification-report accuracy	79%
Report support	7,177 samples

9. Classification Report Interpretation

Detected class	Precision	Recall	F1-score	Support
test	0.00	0.00	0.00	1,477
train	0.79	1.00	0.89	5,700

Detected class	Precision	Recall	F1-score	Support
Accuracy	—	—	0.79	7,177
Macro avg	0.40	0.50	0.44	7,177
Weighted avg	0.63	0.79	0.70	7,177

The report confirms that the recorded model was evaluating the two directory labels **test** and **train**, not the seven facial expressions. The zero precision/recall/F1 for the test class is consistent with the model strongly predicting the train directory label.

10. Confusion Matrix

The notebook generates a confusion matrix using sklearn.metrics.confusion_matrix and visualizes it with a Seaborn heatmap. Because the loaded class_names were ['test', 'train'], the matrix in the recorded run represents those two directory labels rather than the seven facial expressions.

11. Model Saving and Inference

The trained model is saved as **face_expression_cnn_final.keras**. The notebook also contains a Google Colab upload workflow that accepts an image, resizes it to 48×48 grayscale, runs model.predict(), selects the highest-probability class with argmax, and displays the predicted class and confidence.

12. Important Technical Issue and Corrective Action

The most important issue in the current notebook is the dataset root supplied to image_dataset_from_directory. The dataset root contains two folders—train and test—so TensorFlow interprets those as classes. The actual expression classes are one directory level deeper.

For a genuine seven-expression experiment, the training dataset should be loaded from the nested training directory and the test dataset from the nested test directory. Based on the directory structure recorded in the notebook, the intended paths are conceptually:

```
/root/.cache/kagglehub/datasets/aadityasinghal/facial-expression-dataset/versions/1/train/train
/root/.cache/kagglehub/datasets/aadityasinghal/facial-expression-dataset/versions/1/test/test
```

After correcting the paths, the class names should be the seven expression folders, and the model's final Dense layer should have seven output units. The model should then be retrained and the new accuracy, classification report, and confusion matrix should be reported as the actual facial-expression results.

13. Recommended Correct Data Pipeline

A clean evaluation design is: use the provided training directory for model training, use a validation split from the training data (or a dedicated validation set), and keep the provided test directory completely separate for final testing. This avoids using the same images for both training and final evaluation.

Because the dataset is imbalanced—especially the disgust class compared with happy—the corrected experiment should also report per-class precision, recall and F1-score, rather than relying only on overall accuracy.

14. Strengths of the Project

• Uses a CNN architecture appropriate for image classification.

• Uses grayscale 48×48 image inputs, reducing computational cost.

• Includes data augmentation to improve robustness.

• Uses Batch Normalization and Dropout to support stable training and reduce overfitting.

• Uses Early Stopping and learning-rate reduction callbacks.

• Includes classification-report and confusion-matrix evaluation.

• Saves the trained model in Keras format.

• Includes an image-upload inference workflow.

15. Limitations

• The recorded run does not perform seven-class facial-expression recognition because of the dataset-path issue.

• The recorded 79.42% validation accuracy therefore cannot be treated as seven-expression recognition accuracy.

• The dataset has substantial class imbalance, particularly for disgust.

• The notebook does not record a final seven-class test-set score.

• The single-image upload cell was present, but the saved notebook output shows that the upload widget needed to be rerun in the active Colab browser session.

16. Future Improvements

• Correct the train/test directory paths and retrain the model.

• Evaluate on the dedicated test set after training is complete.

• Use class weights or balanced sampling to address class imbalance.

• Experiment with stronger CNN architectures and hyperparameter tuning.

• Compare the custom CNN with transfer-learning models such as MobileNetV2 or EfficientNet.

• Add precision, recall, F1-score and per-class confusion analysis to the final report.

• Add face detection/cropping before expression classification for real-world webcam images.

• Build a real-time webcam application after the seven-class model has been correctly validated.

17. Conclusion

This project demonstrates the complete workflow of a CNN image-classification system in Google Colab: dataset acquisition, directory inspection, image loading, preprocessing, augmentation, CNN construction, training callbacks, evaluation, model saving, and image inference. The notebook recorded 79.42% validation accuracy, but the recorded class labels reveal that the experiment classified the dataset's train/test directories rather than the seven expressions. Correcting the dataset paths and retraining is therefore the essential next step before presenting the project as a facial-expression recognition model.

18. Project Reproducibility Notes

The uploaded notebook records TensorFlow 2.20.0 and a T4 GPU in Google Colab. The Kaggle dataset was downloaded through kagglehub. The report intentionally preserves the notebook's recorded results rather than replacing them with an estimated or assumed result.